

```
*[3]: celsiusAFahrenheit :: Double -> Double
celsiusAFahrenheit c = (c * 1.8) + 32
procesarLecturas celsiusAFahrenheit [0,12,20]

[32.0,53.6,68.0]
```

```

•[7]: crearFiltro :: (a -> Bool) -> ([a] -> [a])
      crearFiltro f = filter f
      soloimpares = crearFiltro odd
      soloimpares [1..20]

```

Eta reduce

Found:	Why Not:
<pre>crearFiltro f = filter f crearFiltro = filter [1,3,5,7,9,11,13,15,17,19]</pre>	

```
[9]: enRango :: Int -> Int -> (Int -> Bool)
    enRango limiteInferior limiteSuperior = \x -> x >= limiteInferior && x <= limiteSuperior
    entrely10 = enRango 1 10
    entrely10 7
    entrely10 12
```

Redundant lambda

Found:	Why Not:
<pre>enRango limiteInferior limiteSuperior = \ x -> x >= limiteInferior && x <= limiteSuperior</pre>	<pre>enRango limiteInferior limiteSuperior x = x >= limiteInferior && x <= limiteSuperior</pre>
True	
False	

```
*[11]: potenciador :: Int -> (Int -> Int)
potenciador exponente = \x -> x ^ exponente
exp4 = potenciador 4
exp4 4
```

Redundant lambda

Found: `potenciador exponente = \ x -> x ^ exponente` **Why Not:** `potenciador exponente x = x ^ exponente`

Avoid lambda using `infix`

Found:	Why Not:
<code>\ x -> x ^ exponente(^ exponente)</code>	

```

* [14]: import Data.List (sortOn)

type Estado = (Int, Int)
type Nodo = Estado
type Costo = Int

capA, capB :: Int
capA = 5
capB = 3

transferirAB :: Estado -> Estado
transferirAB (a, b) = (a - cantidad, b + cantidad)
  where cantidad = min a (capB - b)

transferirBA :: Estado -> Estado
transferirBA (a, b) = (a + cantidad, b - cantidad)
  where cantidad = min b (capA - a)

vecinos :: Estado -> [(Estado, Costo)]
vecinos (a,b) =
  [ ((capA, b), 1)      -- Llenar jarra A
  , ((a, capB), 1)      -- Llenar jarra B
  , ((0, b), 1)         -- Vaciar jarra A
  , ((a, 0), 1)         -- Vaciar jarra B
  , (transferirAB (a,b), 1) -- Verter A->B
  , (transferirBA (a,b), 1) -- Verter B->A
  ]

heuristica :: Estado -> Costo
heuristica (a, _) = abs (a - 4)

aStar :: (Nodo -> [(Nodo, Costo)]) -> (Nodo -> Costo) -> Nodo -> Nodo -> [(Nodo, Costo)]
aStar sucesores heuristica inicio meta = buscar [(inicio, 0)] []
  where
    buscar [] _ = []
    buscar ((nodo, costo):cola) visitados
      | nodo == meta = [(nodo, costo)]
      | nodo `elem` visitados = buscar cola visitados
      | otherwise =
        let nuevos = [ (v, costo + c) | (v, c) <- sucesores nodo, v `notElem` visitados ]
            ordenados = sortOn \(v, c) -> c + heuristica v (cola ++ nuevos)
        in (nodo, costo) : buscar ordenados (nodo : visitados)

let inicio = (0, 0)
let meta = (4, 0)
let solucion = aStar vecinos heuristica inicio meta
putStrLn "Secuencia de estados y costos para medir exactamente 4 litros:"
mapM_ print solucion

```

Secuencia de estados y costos para medir exactamente 4 litros:

```

((0,0),0)
((5,0),1)
((5,3),2)
((2,3),2)
((0,3),1)
((3,0),2)
((3,3),3)
((2,0),3)
((5,1),4)
((0,2),4)
((5,2),5)
((4,3),6)
((4,0),7)

```

```
Untitled.ipynb
[ ]: ¿Cuál fue el tiempo total invertido en el laboratorio por cada uno de ustedes? (Horas/Hombre)
Invertimos 2 horas por integrante

¿Cuál es el estado actual del laboratorio? ¿Por qué?
Finalizado, por que logramos terminar todos los ejercicios

¿Cuál consideran fue el mayor logro? ¿Por qué?
Aprender mas sobre Haskell y de como resolver problemas usandolo, por que no puede llegar a ser util

¿Cuál consideran que fue el mayor problema técnico? ¿Qué hicieron para resolverlo?
Que el codigo sirviera y no diera error, lo resolvimos preguntadole al profesor

¿Qué hicieron bien como equipo? ¿Qué se comprometen a hacer para mejorar los resultados?
Nos dividimos el trabajo satisfactoriamente para que cada uno aportara al desarrollo del laboratorio, nos comprometemos a finalizarlo un poco mas antes

¿Qué referencias usaron? ¿Cuál fue la más útil? Incluyan citas con estándares adecuados.
El profesor y la presentacion del profesor
```